

Full live demo walkthrough (click-by-click)

Read this straight through, live, in front of the professor. Every step names the exact thing to type or click, what should appear on screen, and a short line to say out loud. **Total spoken time target: ~9 minutes** (per-step estimates below; padding included for questions).

Assumes: Windows or any OS with Python 3.8+, this repo cloned, and `pip install -r requirements.txt` already run. If not, run that first — it's not counted in the demo time.

1. Pre-demo setup (before the professor arrives — not part of the timed demo)

Do this once, quietly, before anyone is watching.

1.1 Open a terminal in the project root and run the environment reset script — **recommended even if you think certs already exist**, since you'll likely be on a fresh or different machine for the actual presentation, where `certs/` won't exist at all yet:

```
python demo/reset_environment.py
```

This reports (and asks before killing) anything already listening on the server's port, then (re)generates `certs/server.crt / server.key` and prints the fingerprint of the cert it just wrote. **Note that fingerprint down** — you'll confirm it matches the server's own startup line in the next step. (If you'd rather generate certs manually instead: `python certs/generate_certs.py`, skippable if `certs/server.crt` and `certs/server.key` already exist from a previous run on this machine.)

1.2 In that same terminal, start the relay server:

```
python server/server.py
```

Confirm it printed a line like this before moving on (the port will read `5000` unless you passed `--port`, and the fingerprint will be whatever `reset_environment.py` just printed — **these two must match**; if they don't, see ["Troubleshooting: certificate mismatch"](#) in the main README):

```
[*] Server listening on 127.0.0.1:5000 over TLS (cert fingerprint: XXXX XXXX XXXX XXXX) (routes handshake/chat envelopes; never sees plaintext app secrets or session keys)
```

Leave this terminal running and visible for the whole demo — minimize it to a corner of the screen; you'll point at it in step 5 to show it never prints anyone's message content.

1.3 Quick sanity check (optional but recommended): run `pytest tests/ -v` once and confirm it ends with `105 passed`. This is not shown to the professor; it's your own confidence check that nothing is broken before you start.

2. Launch two GUI clients — ~30 sec

2.1 Open a second terminal, at the project root, and run:

```
python gui/chat_gui.py
```

A dark-themed window titled **"Secure Chat (GUI)"** appears, showing a login card labeled "Secure Chat" with fields for Server host, Server port, Username, Peer username, and Password. Drag this window to the **left half** of the screen.

2.2 Open a third terminal and run the exact same command again:

```
python gui/chat_gui.py
```

A second, identical window appears. Drag this one to the **right half** of the screen, so both are visible side by side, with the server terminal from step 1.2 visible somewhere below or behind them.

Say: "This is the same client logic as our terminal version — the GUI just wraps it, so what you're about to see is the real protocol, not a mockup."

3. Register user 1 (left window) — ~45 sec

3.1 In the **left** window: click the **Server host** field — it's pre-filled with `127.0.0.1`, leave it as-is. Click the **Server port** field — pre-filled with `5000`, leave it as-is.

3.2 Click the **Username** field and type: `alice`

3.3 Click the **Peer username** field and type: `bob`

3.4 Click the **Password** field and type any password, e.g. `demo-pass-1234` (it will show as dots as you type).

3.5 Click the **Register** button.

Expected on screen: the login card is replaced by the chat screen. At the top you see `alice ↔ bob`, and the **Conversation** panel shows a centered gray line reading `Your key fingerprint: XXXX XXXX XXXX XXXX` (16 hex characters in 4 groups). The **Wire Log** panel on the right starts filling with lines like `--> register: username=alice, ...` and `<-- register_result: success=True, ...`.

Say: "Registering just created a fresh RSA-2048 identity for alice — the private key never leaves this machine; only the public key went to the server, and you can see that exact envelope in the wire log on the right."

4. Register user 2 (right window) — ~45 sec

4.1 In the **right** window, repeat exactly: click **Username**, type `bob`; click **Peer username**, type `alice`; click **Password**, type any password (can be different from alice's — they're independent accounts), e.g. `demo-pass-5678`.

4.2 Click **Register**.

Expected on screen: within a couple of seconds, **both** windows' Conversation panels show a centered line: `Secure session established with <other username>`. The top-right of each window now shows a highlighted strip: **"s fingerprint: XXXX XXXX XXXX XXXX"**. The Wire Log on both sides shows `handshake_init` / `handshake_response` lines with truncated `pubkey=...` and `sig=...` values.

Say: "The moment both of us were online, the two clients automatically ran an ECDH handshake — signed with our RSA keys so the server can't substitute its own key in the middle, which we'll prove directly in demo 3 later."

5. Compare fingerprints out loud — ~45 sec

5.1 Point at the highlighted fingerprint strip in the **left** (alice's) window, top-right corner. Read the 16 hex characters out loud, e.g. *"seven, four, six, B..."*

5.2 Point at the same strip in the **right** (bob's) window and read it too.

Expected: the fingerprint alice's window shows for "bob" is **character-for- character identical** to the fingerprint bob's own window shows for himself as "Your key fingerprint" earlier — and vice versa.

Say: "If a relay server — or anyone — tried to slip in a different key mid- handshake, these two fingerprints wouldn't match, and we'd catch it just by reading them aloud to each other, independent of any code running. This is the exact human backstop our design doc calls 'check key fingerprints.'"

6. Exchange messages both directions — ~90 sec

6.1 In the **left** (alice) window, click the message entry field at the bottom and type: `hi bob, this is alice`. Click the **Send** button (or press Enter).

Expected: a teal, right-aligned bubble with that text and a timestamp appears in alice's own Conversation panel. In bob's (right) window, a dark-gray, left-aligned bubble with the same text appears. In **both** Wire Log panels, a new line appears: `--> chat: from=alice nonce=... ciphertext=... tag=... (alice's side) / <-- chat: from=alice nonce=... ciphertext=... tag=... (bob's side)` — truncated base64, never the plaintext.

6.2 In the **right** (bob) window, type: `hey alice, got it` and click **Send**.

Expected: mirrored — teal bubble on bob's side, gray bubble on alice's side, chat envelope lines in both wire logs.

6.3 Send one more each way (any text) so there are at least 2–3 message bubbles each direction visible before moving on.

6.4 In the **right** (bob) window, **click one of alice's gray bubbles**. A small dark popup titled **Security receipt** opens. Read out: the sender and their fingerprint, **AES-GCM tag ✓ verified**, **RSA signature ✓ verified**, **Timestamp ✓ fresh (N s old)**, **Chain position: seq #1 ✓ links to the previous message**, and the truncated nonce and record SHA-256. Press Esc or **Close**. Then click one of bob's own teal bubbles to show the sent version ("✓ signed with your private key").

Say: "Every message carries its own receipt: not 'trust us, it's encrypted', but exactly which checks this specific message passed. Everything on it was verified by the client before the message was ever displayed."

Say: "Notice the Conversation panel reads like an ordinary chat app — that's the point. Now look at the Wire Log: every message we just read in plain English exists on the network only as this — nonce, ciphertext, tag. Nobody watching this connection, including our own relay server, can read what we just said to each other."

7. (Optional) Open the Wireshark capture for a static packet view — ~60 sec

Skip this step if short on time; it's a supplement to the live demo, not required.

7.1 Open Wireshark. **File** → **Open...**, navigate to and select `demo/capture_normal_session.pcapng`, click **Open**.

7.2 In the display filter bar at the top, type `tls` and press Enter to show only TLS-layer packets.

If the **Protocol** column shows `RSL` instead of `TLS`, Wireshark is guessing from the port number (5000 is officially a GSM "RSL" port). Right-click any packet → **Decode As...** → set **Current** to `TLS` → **Save** → **OK**. Do this once before the demo; Wireshark remembers it.

The capture holds two connections (alice's and bob's), each starting with its own TLS handshake.

7.3 Click the first packet whose **Info** column reads `Client Hello (SNI=localhost)`. In the packet-detail pane below, expand **Transport Layer Security** → **Handshake Protocol: Client Hello** to show the TLS handshake beginning the connection, and point at **Extension: server_name** → `localhost`: the server name the client is asking for.

7.4 Click the next packet, whose **Info** column starts `Server Hello`. Expand **Transport Layer Security** → ... → **Server Hello** and point at **Extension: supported_versions** → `TLS 1.3` and **Cipher Suite: TLS_AES_256_GCM_SHA384**.

Say: "The server's certificate is in this packet too, but TLS 1.3 encrypts it, so even the certificate is hidden from an observer. Everything after this Server Hello is encrypted."

7.5 Right-click any packet after the handshake completes (**Info** column reads `Application Data`) → **Follow** → **TCP Stream**. Point at the resulting hex/ASCII pane.

Expected: the reassembled stream is unreadable binary — no `{"type": "login"...}`, no readable text anywhere. To prove it, close that window, type `frame contains "alice"` into the display filter and press Enter: **zero packets** match, even though alice registered and chatted in this session. The same is true for `frame contains "password"` and for any of the message text.

Say: "This is a static capture of exactly the kind of session we just ran live. Every one of these 'Application Data' packets is one of our JSON envelopes — but from the outside, it's indistinguishable from noise. That's TLS doing its job on top of everything else we've already shown."

8. Run the tamper-detection demo — ~60 sec

8.1 In a fourth terminal (or reuse the one from step 1.3), at the project root, run:

```
python demo/run_tamper_demo.py
```

8.2 Let it run to completion (a few seconds). **Expected final output:**

```
=====
RESULTS
=====
[PASS] Normal message displayed correctly.
[PASS] Confirmed the malicious relay actually flipped a byte (this wasn't a no-op).
[PASS] Tampered message content was NEVER displayed to bob.
[PASS] Bob's client printed a clear tamper-detection warning.

=====
DEMO 1 PASSED
=====
```

Above that, point at the line reading `[!] WARNING: message from ... failed authentication (tampered or wrong key) -- discarded.`

Say: "This script starts its own throwaway server that deliberately flips one bit of a real message's ciphertext in transit — simulating a compromised relay. AES-GCM's authentication tag catches it immediately; the tampered content is never shown, only this warning."

9. Run the replay-protection demo — ~60 sec

9.1 Run:

```
python demo/run_replay_demo.py
```

9.2 Expected final output:

```
=====
RESULTS
=====
[PASS] Confirmed the malicious relay actually double-sent the envelope.
[PASS] The message content was displayed exactly once (1 occurrence(s)), not twice.
[PASS] Bob's client printed a clear duplicate/replay rejection warning for the second delivery.

=====
DEMO 2 PASSED
=====
```

Point at the line above reading `[!] WARNING: duplicate message detected from ... (same nonce seen before) -- rejected as a replay -- discarded.`

Say: "Here the malicious relay captures one real, validly-signed message and resends the exact same packet a second time — a classic replay attack. Even though it's byte-for-byte identical to a message that was already accepted once, the timestamp-plus-nonce tracking on the receiving side catches the duplicate."

10. Run the MITM handshake demo — ~75 sec

10.1 Run:

```
python demo/run_mitm_handshake_demo.py
```

10.2 Expected final output:

```

=====
RESULTS
=====
[PASS] Confirmed the malicious relay actually attempted the key swap.
[PASS] Bob's client derived NO session key from the forged handshake -- the MITM attempt was rejected.
[PASS] Bob's client printed a clear handshake-abort warning naming the possible MITM.

[*] For comparison -- what a human reading fingerprints aloud would see:
    Real demo_alice_....'s RSA fingerprint (what bob actually has on file):
        <16 hex chars>
    Attacker's RSA fingerprint (...):
        <different 16 hex chars>
[PASS] These fingerprints differ -- a human comparing them out-of-band (voice call, in person) would also catch this
attacker, independent of any automated check.

=====
DEMO 3 PASSED
=====

```

Point at the `HANDSHAKE ABORTED` line above, and then at the two different fingerprints printed at the bottom.

Say: "This is the attack our fingerprint step in part 5 defends against, proven directly: the relay tries to substitute its own key during the handshake itself — the classic unauthenticated Diffie-Hellman man-in-the-middle. Because we sign the handshake with our RSA identities, the victim's client refuses to derive a session key at all. And even in a hypothetical where that check somehow failed, you can see the resulting fingerprint wouldn't match — the human check from part 5 is a real, independent backstop, not just decoration."

11. Run the dropped-message demo — ~60 sec

11.1 Run:

```
python demo/run_drop_demo.py
```

11.2 Expected output (middle of the run, then the result):

```

--- STEP 2: alice sends message #2 -- but the relay silently DROPS it ---
[MALICIOUS RELAY] silently dropped a valid chat message (it is never forwarded, and nobody is told).

--- STEP 3: alice sends message #3 -- delivered; bob's chain check fires ---
[!] WARNING: 1 message(s) missing before #3 (expected #2): possible deletion by the relay.
...
[PASS] Bob's client detected the gap: '1 message(s) missing ... possible deletion by the relay'.
...
DEMO 4 PASSED

```

Point at the `[MALICIOUS RELAY] silently dropped` line, then at the `WARNING: 1 message(s) missing` line two steps later.

Say: "Encryption and signatures prove a message is real, but they can't tell you about a message that never arrives. Here a compromised relay silently drops a 'STOP, allergic reaction' order. Because every message carries a sequence number and the hash of the previous one, inside the signed payload, the next message exposes the hole immediately. WhatsApp or Signal wouldn't surface this; for a hospital or a command chain, it's the difference that matters."

12. Export signed evidence and verify it independently — ~90 sec

12.1 In either GUI window, click **Export Evidence** (top bar, left of the fingerprint strip). A notice appears in the Conversation panel: `Evidence exported: exports/evidence_<you>_<peer>_<time>.json`, with the exact verify command underneath. (The file is gitignored.)

12.2 Run the scripted version, which also demonstrates tampering:

```
python demo/run_evidence_demo.py
```

12.3 Expected output: the verifier report for the genuine file ends with

```
RESULT: VALID ✓ every signature, chain link and the export signature checked out.
```

then, after the demo edits one word (`Approve` → `Reject`) in record #1 of a copy:

```
✗ #1 ...: signature -- RSA signature does NOT verify (message or chain fields were altered, or wrong key)
✗ #1 ...: record hash -- record_hash does not match the record's content
✗ #3 ...: chain -- prev_hash does not link to the previous record: ...
✗ exporter signature by ... is INVALID -- the file was edited after export ...
RESULT: INVALID ✗ 4 check(s) failed; first failure: record #1 -- signature.
...
DEMO 5 PASSED
```

To verify a GUI export yourself instead (optional): `python tools/verify_transcript.py exports/<file>.json` . To pin a key you trust, add `--expect-fingerprint "XXXX XXXX XXXX XXXX"` (one of the fingerprints from step 5).

Say: "This is the opposite of WhatsApp's deniability. Every message is signed by its author, so this file proves who said what, and the verifier is a separate program that imports none of our chat code, so an auditor doesn't have to trust us, either user, or the server. Change one word anywhere and it names the exact message and the exact check that failed."

13. Closing statement — ~30 sec

Say: "Going back to our Review 1 problem statement: we set out to build a chat system where the relay server — which has to exist, to actually deliver messages — is never a point where message content, passwords, or private keys are exposed, and where standard network-level attacks — eavesdropping, tampering, replay, and man-in-the-middle — are all defended against, not just encrypted-and-hoped. And beyond what WhatsApp or Signal set out to do, it's an auditable messenger: every message is attributable to its author, a relay silently dropping a message is detected, and a signed evidence file can be verified by a third party. What you've just seen is that working end to end: a real GUI with a security receipt on every message, a real wire log showing exactly what crosses the network, and four live attacks each caught by a specific, testable defense — all backed by 105 automated tests in our repo. Happy to answer questions or run any of this again."

Timing summary

Step	Approx. time
2. Launch two GUIs	0:30
3. Register alice	0:45
4. Register bob	0:45
5. Compare fingerprints	0:45
6. Exchange messages	1:30
7. Wireshark (optional)	1:00
8. Tamper demo	1:00
9. Replay demo	1:00
10. MITM handshake demo	1:15
11. Dropped-message demo	1:00

Step	Approx. time
12. Evidence export + verifier	1:30
13. Closing	0:30
Total (with step 7)	~11:30
Total (without step 7, if short on time)	~10:30